AgroPlus

A Soil- and Weather-Aware Crop Recommendation & Farm Profitability Platform for Bangladeshi Farmers

SE 801 : Software Project Lab–III

Submitted by

Name	**Md. Touhidur Rahman**
BSSE Roll No.	**1434**
Exam Roll	**2322144**
Registration No.	**2021011373**
BSSE Session	**2021-22**

Supervised by

Name	**Dr. Zerina Begum**
Designation	**Professor**
Department	**Institute of Information Technology**
	University of Dhaka

Supervisor Signature: _____

Date of Submission: 11 October, 2026

Table of Contents

1. Introduction **5**

1.1 Purpose of the Document 5

1.2 Problem Statement 5

1.3 Project Scope 6

In Scope (Delivered) 6

Out of Scope (This Iteration) 6

2. Requirements Analysis **7**

2.1 Functional Requirements 7

2.1.1 Account & Authentication 7

2.1.2 Land Profile Management 7

2.1.3 Soil Sampling 7

2.1.4 Crop Recommendation Engine 8

2.1.5 Harvest Economics & Profit Estimation 8

2.1.6 Recommendation History 8

2.1.7 Live Conditions & Forecast 8

2.1.8 AI Farm Assistant (Chat) 9

2.2 Non-Functional Requirements 9

3. Stakeholders **10**

4. Quality Function Deployment (QFD) **11**

4.1 Normal Requirements 11

4.2 Expected Requirements 11

4.3 Exciting Requirements 11

5. Entity-Relationship Design **13**

5.1 Entities and Attributes 13

Farmer (table: farmer) 13

District (table: district) 13

Farm – "Land" (table: farm) 13

Recommendation (table: recommendation) 14

SamplePoint (table: sample_point) 14

RecommendationCrop (table: recommendation_crop) 14

CropEconomics (table: crop_economics — reference table) 15

ChatMessage (table: chat_message) 15

5.2 ER Diagram 16

5.3 Design Rationale 16

6. Component-Level Design 17
6.1 System Design Overview 17
6.1.1 Component Catalog 17
6.1.2 Component Relations 18
6.2 Component Descriptions 18
6.2.1 Auth Component 18
6.2.2 Land Profile Component 19
6.2.3 Soil Sampling & Recommendation Pipeline 19
6.2.4 ML Recommendation Engine 20
6.2.5 Crop Economics Knowledge Base 20
6.2.6 Weather Service 21
6.2.7 AI Farm Assistant 21
6.2.8 Persistence Component 21
6.2.9 API Layer (FastAPI Application) 22
6.2.10 Flutter Mobile Client 22
6.3 System Design Diagram 23
7. Interface Design 24
7.1 User Interface Architecture 24
7.2 UI Events and State Transitions 25
7.3 Interface Screenshots 26
8. Behavioural Modelling: State Transition Diagrams 37
8.1 AuthGate 37
8.2 Soil Sampling & Recommendation Pipeline 37
8.3 Chat 38
9. Testing: API Endpoint Testing 39
9.1 Auth — /auth 39
9.2 Districts — /districts 39
9.3 Land Profiles — /farms 40
9.4 Soil Sampling & Recommendations — /farms/{id}/recommendations 41
9.5 AI Farm Assistant — /farms/{id}/chat 41
9.6 Health Check 42
10. User Manual 43
10.1 Creating an Account 43
10.2 Logging In 43
10.3 Adding a Land 43
10.4 Taking a Soil Sample 43
10.5 Viewing Your Crop Recommendation 44

10.6 Checking Weather 44
10.7 Chatting with the Farm Assistant 44
10.8 Viewing Recommendation History 44
10.9 Editing or Deleting a Land 44
10.10 Switching Language 45
11. Conclusion & Future Work 46
11.1 Summary 46
11.2 Known Limitations & Future Work 46

1. Introduction

1.1 Purpose of the Document

This report describes the completed software design, implementation, and testing of **AgroPlus**, a mobile platform that recommends crops to Bangladeshi farmers based on their land's soil chemistry and live weather, and estimates the full cost and profit of growing each recommended crop. It is the final deliverable for the SE 801 project and is intended for the development team, the project supervisor, and evaluators. This report documents what was actually built: the requirements delivered, the data and component design implemented in code, the interface design, behavioural models, a non-technical user manual, and the testing performed against the running system.

The scope of this document covers Quality Function Deployment (QFD) and functional requirements, stakeholder identification, an Entity-Relationship diagram, component-level system design, interface design with real screenshots from the running application, state-transition diagrams, and API endpoint testing.

1.2 Problem Statement

Smallholder and mid-size farmers in Bangladesh routinely decide what to plant based on habit, neighbours' choices, or a retailer's suggestion — rarely on a systematic reading of their own soil's nutrient levels, the current season's weather, or a transparent cost-versus-profit calculation for the specific crop being considered. Three gaps compound this:

- **No accessible soil-to-crop link.** A farmer may be able to get a soil test done, but translating N/P/K/pH numbers into "which of the crops I could grow here will actually do well" requires agronomic expertise most farmers don't have on hand.
- **No transparent economics.** Even when a crop is agronomically sound, farmers rarely have an itemized, sourced estimate of seed cost, fertilizer cost, and total cost against their own land size and budget before committing a season to it — so a technically "good" recommendation can still be a financial mistake.
- **No rotation discipline.** Continuous monocropping or careless back-to-back repetition of the same crop depletes specific soil nutrients and builds up crop-specific pests, but there is no simple tool that remembers a land's recent cropping history and factors it into what gets suggested next.

AgroPlus was built to close these gaps: a mobile app that turns a short soil sample into a ranked (or season-sequenced) crop recommendation grounded in real government and research data, breaks down exactly what that crop will cost and earn on the farmer's own land, enforces a basic crop-rotation rule automatically, and lets the farmer ask a conversational assistant *why* a crop was suggested — in their own language.

1.3 Project Scope

In Scope (Delivered)

- A Flutter mobile application (Android/iOS from one codebase) with English and Bangla localization, backed by a FastAPI REST API.
- Farmer account registration, login, and JWT-based session management with automatic token refresh.
- Land profile management: create, edit, delete, with district, area, soil type, water availability, budget, target season, cropping pattern, and crop-rotation history.
- Tiered soil sampling (1, 3, or 5 points depending on land size) producing a composite N/P/K/pH reading.
- A machine-learning crop recommendation engine (XGBoost + Random Forest ensemble) producing a confidence-ranked top-3, restricted to crops with sourced economics data.
- A double/triple seasonal cropping planner that sequences one crop per growing season (Rabi/Aus/Aman), enforcing a hard "no crop grown more than twice in a row" rotation rule everywhere crops are ranked.
- An itemized, government-data-sourced harvest economics engine: seed cost, per-nutrient fertilizer cost, other costs, gross revenue, and net profit — compared against the farmer's stated budget.
- Live weather conditions and a 7-day forecast per land, from Open-Meteo.
- Persistent, chronological recommendation history per land, with full historical detail.
- An AI farm assistant (OpenAI function-calling chat agent) scoped to one land at a time, grounded exclusively in that land's real stored data, with a persistent per-land conversation and a reset action.

Out of Scope (This Iteration)

- Crop recommendation/economics for crops outside the 8-crop sourced-data vocabulary (rice, maize, chickpea, mungbean, blackgram, lentil, jute, mango).
- Season-specific climate normals for the multi-cropping planner (all seasons in a plan are scored against the same current weather snapshot — see §6.2.4).
- A farmer profile/settings screen (password change, profile photo, notification preferences).
- Automated regression/unit test suites; testing in this iteration was performed by direct API and on-device verification (§9).
- Production deployment, horizontal scaling, and rate-limiting of the AI assistant's API usage.

2. Requirements Analysis

Requirements are grouped by the module of the system they belong to and assigned a unique identifier for traceability. This project has **36 functional requirements** and **9 non-functional requirements**.

2.1 Functional Requirements

2.1.1 Account & Authentication

ID	Requirement
FR-1	The system shall allow a farmer to register an account with name, email, phone, and password.
FR-2	The system shall allow a registered farmer to log in with email and password and receive a JWT access/refresh token pair.
FR-3	The system shall automatically refresh an expired access token using the stored refresh token, retrying the original request, without requiring the farmer to log in again.
FR-4	The system shall allow a farmer to log out, clearing the locally stored session.
FR-5	The system shall support switching the app's display language between English and Bangla at any time, persisted across restarts.

2.1.2 Land Profile Management

ID	Requirement
FR-6	The system shall allow a farmer to create a land profile with name, location, district, area (acres), soil type, water availability, budget, and target season.
FR-7	The system shall allow a farmer to edit any field of an existing land profile.
FR-8	The system shall allow a farmer to delete a land profile and all data associated with it.
FR-9	The system shall allow a farmer to record the land's cropping pattern (single, double, or triple) and the last two crops harvested on it.
FR-10	The system shall restrict every land's visibility and control (view, edit, delete) to the farmer who owns it.

2.1.3 Soil Sampling

ID	Requirement
FR-11	The system shall determine the number and layout of required soil sample points for a land based on its area (1-point, 3-point, or 5-point tiers).
FR-12	The system shall allow a farmer to enter Nitrogen, Phosphorus, Potassium, and pH values for each required sample point.
FR-13	The system shall compute a composite soil reading (the average N/P/K/pH) once every required point for a land's current tier has been saved.
FR-14	The system shall discard an in-progress (incomplete) soil reading if the land's area is edited across a sampling-tier boundary, since the previous point layout no longer applies.

2.1.4 Crop Recommendation Engine

ID	Requirement
FR-15	The system shall generate the top-3 recommended crops for a land's composite soil reading and current live weather, each with a confidence score.
FR-16	The system shall restrict recommendations to a fixed set of crops for which sourced yield/price/cost data exists.
FR-17	The system shall exclude a crop from recommendation if it has already been grown twice in a row on that land, counting the land's recorded crop-rotation history.
FR-18	The system shall, for a land configured for double or triple cropping, generate a season-sequenced planting plan (one crop per growing season) instead of a single ranked list.
FR-19	The system shall substitute the next-best candidate and record a human-readable reason when a season's top-scoring crop is blocked by the rotation rule.
FR-20	The system shall re-generate a land's recommendation every time its composite soil reading is recalculated, discarding the previous result.

2.1.5 Harvest Economics & Profit Estimation

ID	Requirement
FR-21	The system shall display, for any recommended crop, its expected yield, market price, and growing season.
FR-22	The system shall display an itemized cost breakdown (seed cost, per-nutrient fertilizer cost, and other costs) and the total estimated cost for a crop scaled to the land's size.
FR-23	The system shall display the estimated gross revenue and net profit for a crop on a given land.
FR-24	The system shall warn the farmer when a crop's estimated total cost exceeds the land's stated budget, on both the recommendation list and the crop detail screen.

2.1.6 Recommendation History

ID	Requirement
FR-25	The system shall persist every completed soil reading and its resulting recommendation.
FR-26	The system shall display a land's recommendation history in chronological order, newest first.
FR-27	The system shall allow a farmer to view the full detail (composite reading and every ranked or sequenced crop) of any past recommendation.

2.1.7 Live Conditions & Forecast

ID	Requirement
FR-28	The system shall display current temperature, humidity, and rainfall for a land's district, refreshable on demand.
FR-29	The system shall display a 7-day weather forecast (daily high/low temperature and precipitation) for a land's district.

2.1.8 AI Farm Assistant (Chat)

ID	Requirement
FR-30	The system shall provide a conversational assistant scoped to exactly one land at a time.
FR-31	The system shall ground the assistant's answers in the land's actual stored data (soil reading, live weather, saved recommendation, crop economics) rather than general knowledge, by calling backend tools rather than guessing.
FR-32	The system shall allow the assistant to explain why a specific crop was recommended, grounded in a comparison of the land's soil/weather values against that crop's typical training-data range.
FR-33	The system shall allow the assistant to list crop options beyond the saved top-3, honoring the same rotation rule as the main recommendation engine.
FR-34	The system shall persist each land's conversation history and restore it the next time the farmer opens that land's chat.
FR-35	The system shall allow a farmer to clear a land's chat history and start a new conversation, after confirmation.
FR-36	The system shall have the assistant reply in the same language the farmer is writing in.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Live weather conditions, the 7-day forecast, and a land's current recommendation shall each load within a few seconds under normal network conditions.
NFR-2	Usability	A farmer shall be able to create a land profile and reach their first crop recommendation in no more than the minimum required soil-sampling steps for that land's tier, without needing external documentation.
NFR-3	Reliability	If the weather provider or AI provider is unavailable, the system shall preserve already-saved data (e.g. a soil reading) rather than fail the whole operation, and surface a clear error scoped to only the affected section.
NFR-4	Security	Farmer passwords shall be stored as bcrypt hashes, never in plaintext; every API route except registration/login/public lookups shall require a valid JWT bearer token; every land, recommendation, and chat resource shall be scoped to its owning farmer at the database query level.
NFR-5	Data Integrity	The three cost components (seed, fertilizer, other) of a crop's estimated total cost shall always reconcile exactly to the bundled total cost for that crop.
NFR-6	Maintainability	The ML recommendation engine, the chat agent's tool layer, and the weather integration shall be implemented as independently replaceable service modules behind narrow function interfaces.
NFR-7	Portability	The mobile client shall run on Android and iOS from a single Flutter codebase, with English and Bangla localizations maintained in parallel and switchable at runtime.
NFR-8	Traceability	Every number the AI assistant states in a reply shall be attributable to a specific backend tool call result, never fabricated from the language model's own training knowledge.
NFR-9	Data Provenance	Every crop's yield, price, and cost figures shall be traceable to a named public data source (BAMIS, DAM, or BADC), not an estimate invented by the system.

3. Stakeholders

A stakeholder is any individual, group, or organization that affects, or is affected by, the AgroPlus platform. Stakeholders are grouped below as primary (those who directly use or are directly accountable for the system) and secondary (those with an indirect interest, such as providers of underlying data or infrastructure).

Stakeholder	Category	Interest / Role
Smallholder & Mid-Size Farmers	Primary	End users of the app. Create land profiles, submit soil samples, receive crop recommendations and profit estimates, and chat with the farm assistant to decide what to plant.
Agricultural Extension Officers / NGOs	Primary	Could use the app's recommendations and economics as a standardized, data-grounded reference when advising farmers in their area, rather than relying solely on field experience.
Microfinance Institutions / Input Retailers	Secondary	Could reference a farmer's itemized cost breakdown and budget comparison when sizing a seasonal loan or input package.
BAMIS (Bangladesh Agro-Meteorological Information System)	Secondary	Publishes the district-level yield, cost, profit, and fertilizer-recipe data the economics engine is seeded from; accuracy of the app's cost estimates depends on BAMIS's published figures.
DAM (Department of Agricultural Marketing)	Secondary	Publishes the market price data used for both crop selling prices and fertilizer input prices.
BADC (Bangladesh Agricultural Development Corporation)	Secondary	Publishes the seed price data used to compute seed cost per acre.
Open-Meteo	Secondary	Provides the live weather conditions and 7-day forecast the app and recommendation engine depend on.
OpenAI	Secondary	Provides the language model that powers the farm assistant's tool-calling conversation loop; the assistant's availability and cost depend on this provider.
Kaggle Crop Recommendation Dataset Maintainers	Secondary	Source of the training data (N/P/K/temperature/humidity/pH/rainfall → crop label) the XGBoost/Random Forest recommendation models were trained on.

4. Quality Function Deployment (QFD)

Requirements are grouped into three categories — Normal, Expected, and Exciting — by the degree to which their presence or absence affects farmer satisfaction.

4.1 Normal Requirements

Fundamental features users explicitly need; satisfaction rises roughly in proportion to how well they are implemented, and their absence would make the product unusable for its stated purpose.

Requirement	Description	Implemented Via
Land Profile Management	Create, edit, and delete a land with its agronomic and financial details.	`POST/PATCH/DELETE /farms`
Soil Sampling	Tiered point collection producing a composite N/P/K/pH reading.	`sampling_service.py, PUT /farms/{id}/recommendations/current/points/{label}`
Crop Recommendation Engine	ML-ranked top-3 crops for a land's soil and weather.	`recommendation_service.get_recommendation`
Harvest Economics & Profit Estimation	Itemized, sourced cost breakdown and profit for a crop.	`get_crop_detail, crop_harvest_detail_screen.dart`

4.2 Expected Requirements

Features not always explicitly requested but assumed by users familiar with a farm-advisory tool; their quality of implementation has a direct, proportional effect on satisfaction.

Requirement	Description	Implemented Via
Recommendation History	Chronological record of every past completed recommendation.	`GET /farms/{id}/recommendations, recommendation_history_screen.dart`
Live Conditions & 7-Day Forecast	Current weather plus a short-range outlook for planning.	`weather_service.py, weather_forecast_card.dart`
Bangla / English Localization	Full UI available in the farmer's preferred language.	`lib/l10n/*, locale_provider.dart`

4.3 Exciting Requirements

Features that go beyond what farmers would typically expect from a crop-advisory app. Their presence differentiates AgroPlus from a static lookup table; their absence would not be perceived as a deficiency.

Requirement	Description	Implemented Via

AI Farm Assistant (Grounded Chat)	Conversational assistant that only states numbers it fetched from the land's own data, never invented.	`chat_agent_service.py (OpenAI function calling)`
Multi-Cropping Rotation Planning	Season-sequenced double/triple cropping plan with an enforced "no crop more than twice in a row" agronomic rule.	`build_multi_crop_plan, crops_at_rotation_limit`
Recommendation Explanation	"Why this crop?" — compares the land's actual values against the crop's typical range instead of a black-box score.	`get_recommendation_explanation`
Budget-Aware Warnings	Flags a crop as unaffordable against the farmer's own stated budget, not just profitable in the abstract.	`recommendation_list_screen.dart, crop_harvest_detail_screen.dart`

5. Entity-Relationship Design

AgroPlus persists data in PostgreSQL via SQLAlchemy ORM models (`backend/app/models/`), with schema evolution managed through Alembic migrations. There are eight entities: one for farmer accounts, one static reference table, and six describing a land and its recommendation lifecycle.

5.1 Entities and Attributes

Farmer (table: farmer)

Field	Type	Constraints / Notes
id	UUID	Primary key
name	String	Not null
email	String	Unique, indexed
phone	String	Not null
password_hash	String	Bcrypt hash, never stored in plaintext
home_address	String, nullable	
created_at	DateTime	Default now (UTC)

District (table: district)

Field	Type	Constraints / Notes
id	UUID	Primary key
name	String	Unique, indexed — all 64 Bangladesh districts seeded
center_lat / center_lng	Float, nullable	District centroid, used to query the weather provider

Farm – "Land" (table: farm)

Field	Type	Constraints / Notes
id	UUID	Primary key
farmer_id	UUID (FK → farmer.id)	Cascade delete
farm_name, location	String	Not null
district_id	UUID (FK → district.id)	
size_acres	Float	Drives the soil-sampling tier
soil_type	String	CHECK: Clay, Loamy, Sandy, Silty
water_availability	String	CHECK: Irrigated, Rain-fed, Limited

budget	Float	Farmer's stated budget (BDT), compared against estimated cost
target_season	String	CHECK: Rabi, Aus, Aman
cropping_pattern	String	CHECK: single, double, triple — default 'single'
previous_crop_1 / previous_crop_2	String, nullable	Last two crops harvested, feeds the rotation rule
created_at	DateTime	

Recommendation (table: recommendation)

Field	Type	Constraints / Notes
id	UUID	Primary key
farm_id	UUID (FK → farm.id)	Cascade delete
status	String	CHECK: in_progress, complete
composite_n / p / k / ph	Float, nullable	Averaged across required sample points
created_at	DateTime	

SamplePoint (table: sample_point)

Field	Type	Constraints / Notes
id	UUID	Primary key
recommendation_id	UUID (FK → recommendation.id)	Cascade delete
point_label	String	e.g. center, corner_1, quadrant_3 — unique per recommendation
n, p, k, ph	Float	Not null

RecommendationCrop (table: recommendation_crop)

One row per crop in a recommendation — either a top-3 ranked alternative (single cropping) or a season-sequenced plan entry (double/triple cropping).

Field	Type	Constraints / Notes
id	UUID	Primary key
recommendation_id	UUID (FK → recommendation.id)	Cascade delete
rank	Integer	Unique per recommendation; top-3 order, or plan order (1st/2nd/3rd crop of the year)
crop	String	Not null
confidence	Float	Model confidence score
yield_estimate, net_profit, total_cost	Float, nullable	Null only if the crop has no economics row

season	String, nullable	Set only for double/triple plan rows (Rabi/Aus/Aman)
rotation_note	String, nullable	Set only when a rotation-blocked crop was substituted

CropEconomics (table: crop_economics — reference table)

National-average yield/price/cost per crop (and market variety), seeded from BAMIS, DAM, and BADC — not farmer-editable. Joined to a recommendation by crop name at query time rather than a foreign key, since the ML model only ever outputs a generic crop name.

Field	Type	Constraints / Notes
id	UUID	Primary key
crop, variety	String	Unique together — e.g. rice/boro_medium, rice/amon_hybrid_coarse
yield_kg_per_acre, price_bdt_per_kg	Float	Not null
cost_bdt_per_acre	Float, nullable	BAMIS's own bundled per-acre cost estimate
seed_cost_bdt_per_acre	Float, nullable	BADC/DAM seed price × BAMIS seed rate
fertilizer_cost_bdt_per_acre, fertilizer_breakdown	Float / JSON, nullable	BAMIS fertilizer recipe × real DAM fertilizer prices, itemized per nutrient
other_cost_bdt_per_acre	Float, nullable	Remainder of cost_bdt_per_acre after seed + fertilizer (labor, irrigation, land prep)
season_label	String, nullable	Verbatim BAMIS cultivation-season text
season	String, nullable	season_label classified into Rabi/Aus/Aman for the multi-cropping planner
source, as_of_date	String / Date, nullable	Data provenance

ChatMessage (table: chat_message)

Field	Type	Constraints / Notes
id	UUID	Primary key
farm_id	UUID (FK → farm.id)	Cascade delete — one thread per land
role	String	CHECK: user, assistant
content	Text	Not null
created_at	DateTime	Orders the conversation

5.2 ER Diagram

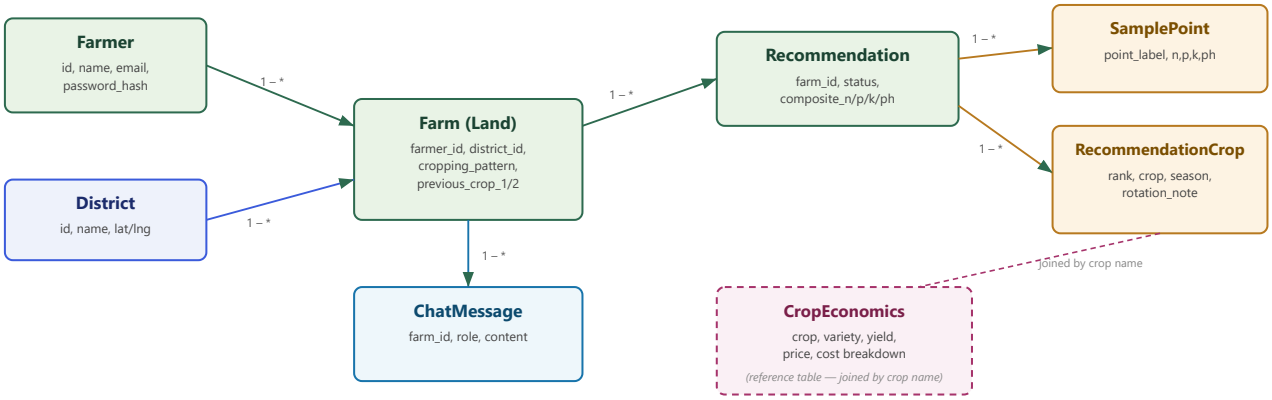


Figure 1: Entity-Relationship Diagram

5.3 Design Rationale

`RecommendationCrop` is a single table shared by both single-cropping (top-3 ranked alternatives) and double/triple cropping (a season-sequenced plan), discriminated implicitly by whether `season` is null. This was chosen because both modes share the same lifecycle (belongs to one recommendation, has a crop/confidence/economics shape) and the same read path (the client fetches `RecommendationOut.top_crops` regardless of pattern and decides how to render it). A shared table avoids a second parallel schema for what is structurally the same kind of row, at the cost of two nullable columns that only apply to one mode.

`CropEconomics` is deliberately not foreign-keyed to `RecommendationCrop`. The ML model only ever outputs a generic crop name (e.g. "rice"), never a market variety, so the join happens by crop name at query time — letting `crop_economics` stay a government-data-sourced reference table that is reseeded independently of any farmer's data, never written to by farmer activity.

6. Component-Level Design

Class-Responsibility-Collaborator (CRC) cards assume an object-oriented paradigm. AgroPlus's backend is built as function-based, modular-procedural FastAPI services — business logic lives in standalone functions, state is managed at the module or database level, and SQLAlchemy model classes are passive data containers without behavioural responsibilities. A component-based system design is used instead, mapping responsibilities, core functions, and inter-service collaborations to the system's actual unit of decomposition: routers, services, and the Flutter client.

6.1 System Design Overview

AgroPlus is organized into ten components: a thin API layer hosting six routers; four backend services (the ML recommendation engine, the crop economics knowledge base, the weather integration, and the AI farm assistant); a persistence layer; and the Flutter mobile client that is the sole consumer of the REST surface.

6.1.1 Component Catalog

Component	Role in the System
Auth Component	Registers and authenticates farmers; issues the JWT access/refresh token pair every other protected route depends on.
Land Profile Component	CRUD for a farmer's land profiles, including cropping pattern and rotation history; also serves live conditions and forecast.
Soil Sampling & Recommendation Pipeline	Collects tiered sample points, computes the composite reading, and triggers the recommendation engine and economics lookups on completion.
ML Recommendation Engine	XGBoost + Random Forest ensemble scoring crops against N/P/K/temperature/humidity/pH/rainfall; also hosts the rotation filter, the multi-crop planner, and the explanation function.
Crop Economics Knowledge Base	Government-sourced (BAMIS/DAM/BADC) yield, price, and itemized cost reference data, reseeded independently via offline scripts.
Weather Service	Fetches current conditions and a 7-day forecast from Open-Meteo for a district's coordinates.
AI Farm Assistant	OpenAI function-calling chat agent, scoped to one land, grounded in that land's data via six backend tools.
Persistence	SQLAlchemy/PostgreSQL durable storage for farmers, lands, recommendations, and chat history, managed via Alembic migrations.
API Layer (FastAPI app)	Hosts all routers and the health check; the single network boundary for the Flutter client.
Flutter Mobile Client	The only consumer of the REST surface; drives land management, sampling, recommendations, and chat across Android/iOS, in English or Bangla.

6.1.2 Component Relations

Component	Talks To	Channel / Mechanism	Direction
Flutter Client	API Layer (all routers)	REST over HTTP (Dio, JWT bearer)	Client → Backend
Auth Component	Persistence	SQLAlchemy ORM (Farmer table)	Auth → Persistence
Land Profile Component	Weather Service	In-process async call	Land → Weather
Soil Sampling Pipeline	Weather Service	In-process async call, on reading completion	Pipeline → Weather
Soil Sampling Pipeline	ML Recommendation Engine	In-process function call (single or plan path)	Pipeline → Engine
ML Recommendation Engine	Crop Economics KB	SQLAlchemy query by crop/variety/season	Engine → KB
Soil Sampling Pipeline	Persistence	SQLAlchemy ORM (Recommendation, RecommendationCrop)	Pipeline → Persistence
AI Farm Assistant	External LLM	OpenAI Chat Completions API (function calling)	Assistant → OpenAI
AI Farm Assistant	ML Engine, Crop Economics KB, Weather Service	In-process tool dispatch, one call per model-requested tool	Assistant → Services
AI Farm Assistant	Persistence	SQLAlchemy ORM (ChatMessage), on every turn	Assistant → Persistence
API Layer	Auth / Land / Pipeline / Assistant	Direct in-process function calls	API → Components
Flutter Client	Persistence (via API Layer)	REST read of history/detail	Client → Backend → Persistence

6.2 Component Descriptions

6.2.1 Auth Component

Source files: `app/routers/auth.py`, `app/core/security.py`, `app/core/dependencies.py`

Purpose: Owns farmer account registration and login, and the JWT dependency every other protected router relies on to identify the caller.

Key Functions / Operations: `register()` — creates a Farmer record and returns a token pair. `login()` — verifies credentials and returns a token pair. `refresh()` — exchanges a valid refresh token for a new access token. `hash_password()` / `verify_password()` — bcrypt hashing.

`create_access_token()` / `create_refresh_token()` / `decode_token()` — JWT issuance and verification. `get_current_farmer()` — the FastAPI dependency imported by every protected router.

Events: `POST /auth/register` and `POST /auth/login` each emit a token pair; `POST /auth/refresh` emits a new access token; `get_current_farmer()` fires on every call into a protected route — a cross-cutting dependency rather than a one-time event.

Relations: Writes/reads the Farmer row through Persistence. Depended on by every other backend component via `get_current_farmer()`.

6.2.2 Land Profile Component

Source files: `app/routers/farms.py`, `app/models/farm.py`, `app/services/sampling_service.py`

Purpose: CRUD for a farmer's land profiles, and the two weather read-throughs (live conditions, 7-day forecast) scoped to a land's district.

Key Functions: `create_farm()`, `update_farm()`, `delete_farm()`, `list_farms()` — standard CRUD, always filtered to the authenticated farmer's own rows. `get_farm_conditions()` / `get_farm_forecast()` — proxy to the Weather Service. `required_points_for_area()` (sampling_service.py) — the single source of truth for the 1/3/5-point sampling tiers.

Events: `PATCH /farms/{id}` with a changed `size_acres` that crosses a tier boundary emits a side effect — any `in_progress` Recommendation for that land is discarded, since its partial points no longer match the new required set.

Relations: Reads/writes Farm rows via Persistence. Calls the Weather Service in-process. Is read by the Soil Sampling Pipeline to determine the active tier.

6.2.3 Soil Sampling & Recommendation Pipeline

Source files: `app/routers/recommendations.py`

Purpose: Collects sample points one at a time, computes the composite reading once the land's tier is satisfied, and — on every completion — triggers either the single-cropping path or the multi-cropping planner, then persists the result.

Key Functions: `upsert_point()` — the central pipeline function: validates the point label against the land's current tier, saves it, and if the required set is now fully satisfied, recomputes the composite N/P/K/pH, fetches live weather, and branches on `farm.cropping_pattern`. `get_current_recommendation()`, `list_recommendations()`, `get_recommendation_detail()` — history reads. `get_recommended_crop_detail()` — proxies to the ML Engine's `get_crop_detail()`.

Events: Every `PUT /farms/{id}/recommendations/current/points/{label}` re-checks tier completion; on completion it emits calls into the Weather Service and the ML Recommendation Engine, then a Persistence write of the new `RecommendationCrop` rows. A weather-provider failure is caught so the composite reading still saves even if crops cannot be generated that attempt.

Relations: Calls the Weather Service and the ML Recommendation Engine in-process. Writes Recommendation/SamplePoint/RecommendationCrop rows to Persistence.

6.2.4 ML Recommendation Engine

Source files: `app/services/recommendation_service.py` , `app/ml_artifacts/*.joblib`

Purpose: Scores every candidate crop against a land's soil and weather values, enforces the crop-rotation rule, builds season-sequenced multi-cropping plans, and explains a recommendation's grounding.

Key Functions: `get_all_crop_scores()` — ensembles XGBoost and Random Forest probabilities (simple average), masked and renormalized to the 8 `SUPPORTED_CROPS` , with an optional `exclude_crops` rotation filter. `get_recommendation()` — the top-3 slice used for single cropping. `crops_at_rotation_limit()` — shared helper computing which crop names already appear twice in a land's history. `build_multi_crop_plan()` — buckets candidates by season (from `CropEconomics.season`), picks the best rotation-safe crop per season, with fallback substitution and a recorded `rotation_note` . `get_profit_estimate()` / `get_crop_detail()` — join a crop's scored result to its economics row(s) for yield/price/cost/profit. `get_recommendation_explanation()` — compares a land's actual feature values to the crop's typical training-data range plus the model's global feature importances.

Events: Called synchronously by the Soil Sampling Pipeline on every reading completion, and by the AI Farm Assistant on-demand per chat tool call — no background job, no polling.

Relations: Queries the Crop Economics Knowledge Base by crop/variety/season. Loaded model artifacts are cached at process start (lazy, memoized) rather than reloaded per call.

> **Known limitation:** the multi-cropping planner scores every season bucket against the *same* current soil + live-weather reading. Which crops are *eligible* per season is accurate (BAMIS-sourced classification); the *ranking* within a season is a same-snapshot approximation, since season-specific historical climate normals per district were out of scope for this iteration.

6.2.5 Crop Economics Knowledge Base

Source files: `app/models/crop_economics.py` ; seeded by `backend/seed_crop_economics.py` and `backend/clean_dam_prices.py`

Purpose: Holds the national-average yield, price, and itemized cost data every profit and explanation figure is grounded in, sourced from three Bangladesh government datasets rather than estimated.

Key Functions: `clean_dam_prices.py` — parses DAM's raw market-price CSV, removes statistical outliers, and aggregates both crop selling prices and fertilizer input prices per kg. `seed_crop_economics.py` — joins BAMIS yield/cost/fertilizer-recipe data (by crop and variety) with DAM prices and BADC seed prices, itemizes fertilizer cost against real per-kg prices, classifies each row's cultivation-season text into Rabi/Aus/Aman, and upserts the result.

Events: Re-run manually, offline, whenever source spreadsheets are updated — not triggered by any farmer action.

Relations: Queried in-process by the ML Recommendation Engine; has no outbound dependencies of its own.

6.2.6 Weather Service

Source files: `app/services/weather_service.py`

Purpose: Fetches current conditions and a 7-day forecast for a district's coordinates from Open-Meteo.

Key Functions: `fetch_current_conditions()` — reads Open-Meteo's `current` block. `fetch_forecast()` — reads the `daily` block for temperature max/min and precipitation sum over 7 days.

Events: Called in-process by the Land Profile Component (on-demand refresh), the Soil Sampling Pipeline (on every reading completion), and the AI Farm Assistant (on tool call) — a `WeatherServiceError` is raised on any HTTP failure and handled distinctly by each caller.

Relations: Talks only to the external Open-Meteo API; has no dependency on any other AgroPlus component.

6.2.7 AI Farm Assistant

Source files: `app/services/chat_agent_service.py`, `app/routers/chat.py`, `app/models/chat_message.py`

Purpose: A conversational assistant scoped to one land, grounded exclusively in that land's real data via six backend tools, with a persistent per-land thread.

Key Functions: `run_chat_agent()` — the tool-calling loop (up to 5 rounds): sends the system prompt plus conversation history to the OpenAI Chat Completions API, dispatches any requested tool calls, and feeds results back until the model returns a final answer. Six tools: `get_soil_reading`, `get_live_weather`, `get_weather_forecast`, `get_current_recommendation`, `get_crop_detail`, `get_other_crop_options`, and `get_recommendation_explanation`. `_build_system_prompt()` — injects the land's static profile (soil type, budget, cropping pattern, rotation history) so the agent doesn't need a tool call for context that's already known.

Events: `POST /farms/{id}/chat` — loads prior turns, runs the loop, persists both the new user message and the reply, returns the reply synchronously (no background job). `GET /farms/{id}/chat/messages` — restores history on chat open. `DELETE /farms/{id}/chat/messages` — clears a thread for "start new conversation."

Relations: Calls the ML Recommendation Engine, Crop Economics Knowledge Base, and Weather Service in-process per tool dispatch. Talks to the external OpenAI API. Writes every turn to Persistence.

6.2.8 Persistence Component

Source files: `app/models/*.py`, `app/db/session.py`, `app/alembic/`

Purpose: The single durable record of farmers, lands, recommendations, and chat history, via SQLAlchemy ORM models against PostgreSQL, with schema changes tracked as versioned Alembic migrations.

Relations: Written to by every other backend component through its own models; has no outbound calls — a storage leaf.

6.2.9 API Layer (FastAPI Application)

Source files: `app/main.py`, `app/routers/*.py`

Purpose: Wires together all six routers (auth, farmers, districts, farms, recommendations, chat) and exposes the health check. A thin composition layer with no business logic of its own.

Relations: The sole entry point used by the Flutter client; delegates in-process to every other backend component.

6.2.10 Flutter Mobile Client

Source files: `lib/core/*, lib/features/{auth,dashboard,farm,chat}/*, lib/l10n/*`

Purpose: The only client of the backend's REST surface. Drives registration/login, land management, soil sampling, recommendation display (single list or season plan), crop economics, and chat — across Android and iOS, in English or Bangla.

Key Functions: `ApiClient` (core/api_client.dart) — Dio instance with a JWT-attaching interceptor and automatic refresh-and-retry on 401. `SecureStorage` — encrypted on-device token storage. Per-feature `ChangeNotifier` providers (`AuthProvider`, `FarmProvider`, `FarmDetailProvider`, `ChatProvider`, `RecommendationHistoryProvider`, `CropDetailProvider`) own all client-side state. One screen widget per concern (dashboard, add/edit land modal, land profile, recommendation list, crop harvest detail, chat, recommendation history/detail).

Relations: Talks exclusively to the API Layer, never directly to any backend service.

6.3 System Design Diagram

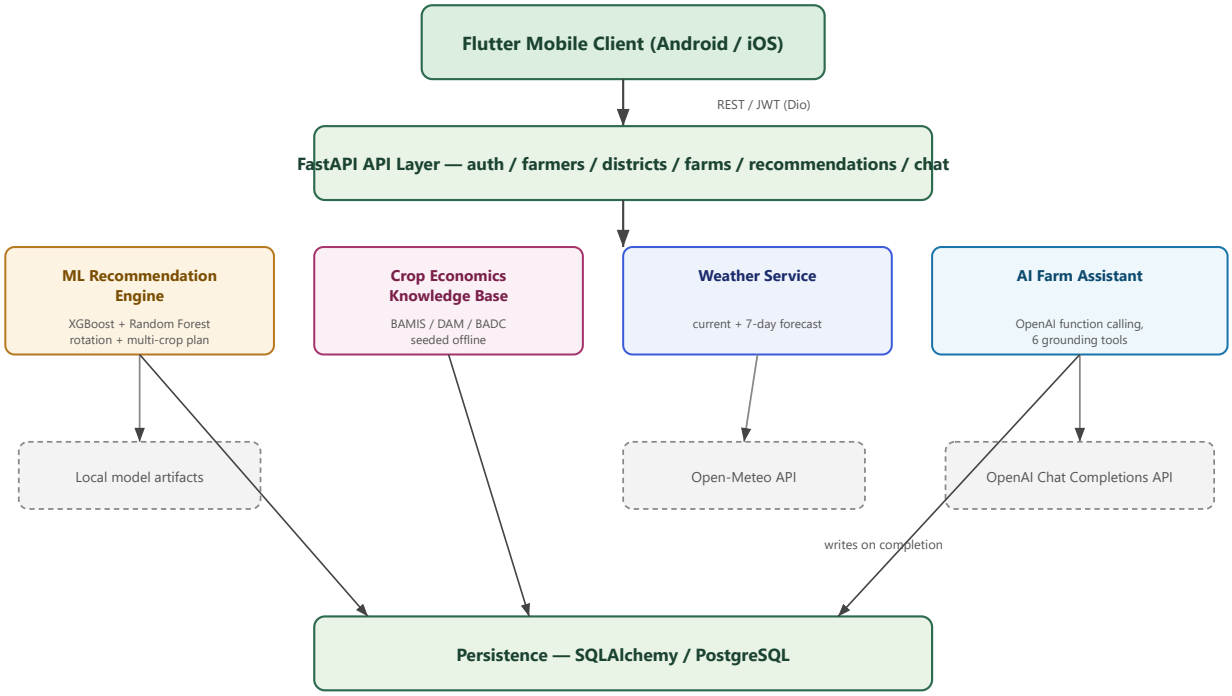


Figure 2: System Design Diagram

7. Interface Design

7.1 User Interface Architecture

The AgroPlus frontend is a Flutter single-page-per-route application following a provider-based (ChangeNotifier) state-management architecture. A startup gate checks the stored session and routes to either the authentication screens or the dashboard. From the dashboard, farmers manage multiple lands, each with its own profile, live conditions, soil sampling grid, crop recommendation (or seasonal plan), and a dedicated, persistent chat thread. Below are the key UI objects and their associated actions.

UI Object	Type	Actions (Operations)
Login / Register Screens	Form Group	Submit login (email + password); submit registration (name, email, phone, password); navigate between the two; switch display language
DashboardScreen	List / Navigation	List all lands with district, area, last-checked time; open a land; add a new land; delete a land (confirm dialog); open the farm picker to chat; log out; switch language
AddFarmModal	Form	Enter/edit all 11 land fields including cropping pattern and the two previous-crop dropdowns; submit as add or edit depending on mode
LandProfileCard	Data Display	View all land fields read-only; open the edit modal
LiveConditionsCard	Data Display	View current temperature/humidity/rainfall; manual refresh
WeatherForecastCard	Data Display	Horizontally scroll a 7-day high/low temperature and precipitation strip
SoilReadingCard / SoilPointModal	Grid / Form	View the point grid (checkmarks for saved points); add or edit N/P/K/pH for a point; view the composite reading once complete
RecommendationCard	Navigation	Open the current recommendation/plan; open recommendation history; open chat for this land
RecommendationListScreen	List	View top-3 (rank badge) or a season plan (season badge); see an over-budget warning chip and a rotation-substitution caption where applicable; open a crop's detail
CropHarvestDetailScreen	Data Display	View season, yield, price, itemized cost breakdown, total cost, gross revenue, net profit, and a budget warning if applicable; open "Why this crop?" into chat
RecommendationHistoryScreen / DetailScreen	List / Data Display	Browse past recommendations chronologically; open one to see its full composite reading and crop list in read-only form
ChatScreen	Chat / Messaging	Send a message; tap a quick-suggestion chip; start a new conversation (confirm dialog, clears history); messages render Markdown (bold, lists)
FarmPickerModal	Selection List	Choose which land to chat about, when entering chat from the dashboard

7.2 UI Events and State Transitions

User Event	Action	UI State Change
Submit Register form	`POST /auth/register`	Form → Loading → tokens stored → Dashboard, or inline validation error
Submit Login form	`POST /auth/login`	Loading → tokens stored → Dashboard, or inline error
App start (session check)	Read stored token	Checking → Dashboard if a token exists, else Login
Any API call returns 401	`POST /auth/refresh`	Request transparently retried with a new access token; session cleared only if refresh also fails
Submit Add/Edit Land form	`POST or PATCH /farms`	Modal closes, dashboard/profile refreshes with the new data
Save a soil sample point	`PUT /farms/{id}/recommendations/current/points/{label}`	Point marked complete; once the tier is fully satisfied, Recommendation card updates from "pending" to showing a result
Tap "Best crops for my land"	Client-side navigation	Opens RecommendationListScreen, rendering rank or season badges per `croppingPattern`
Tap a crop row	`GET /farms/{id}/recommendations/current/crops/{crop}`	Opens CropHarvestDetailScreen with full cost/profit breakdown
Tap "Why this crop?"	Client-side navigation, pre-filled message	Opens Chat, auto-sends the explanation question, agent replies grounded in that crop's comparison
Send a chat message	`POST /farms/{id}/chat`	User bubble appends immediately; typing indicator shows; agent bubble appends on reply (Markdown-rendered)
Tap "New conversation"	`DELETE /farms/{id}/chat/messages`	Confirm dialog → on confirm, message list clears to the initial greeting
Open chat from dashboard	Client-side navigation	FarmPickerModal opens over the dashboard → selecting a land opens Chat scoped to it
Open chat from a land page	Client-side navigation	Chat opens directly, scoped to the current land — no picker

Tap "View history"	`GET /farms/{id}/recommendations`	Opens the chronological history list
Tap a history entry	`GET /farms/{id}/recommendations/{recId}`	Opens read-only detail for that past recommendation
Toggle language	Client-side, persisted locally	Entire UI text switches between English and Bangla immediately
Delete a land	`DELETE /farms/{id}`	Confirm dialog → on confirm, the land and all its data are removed; dashboard list refreshes

7.3 Interface Screenshots

The figures below are captured directly from the running application on an Android emulator.

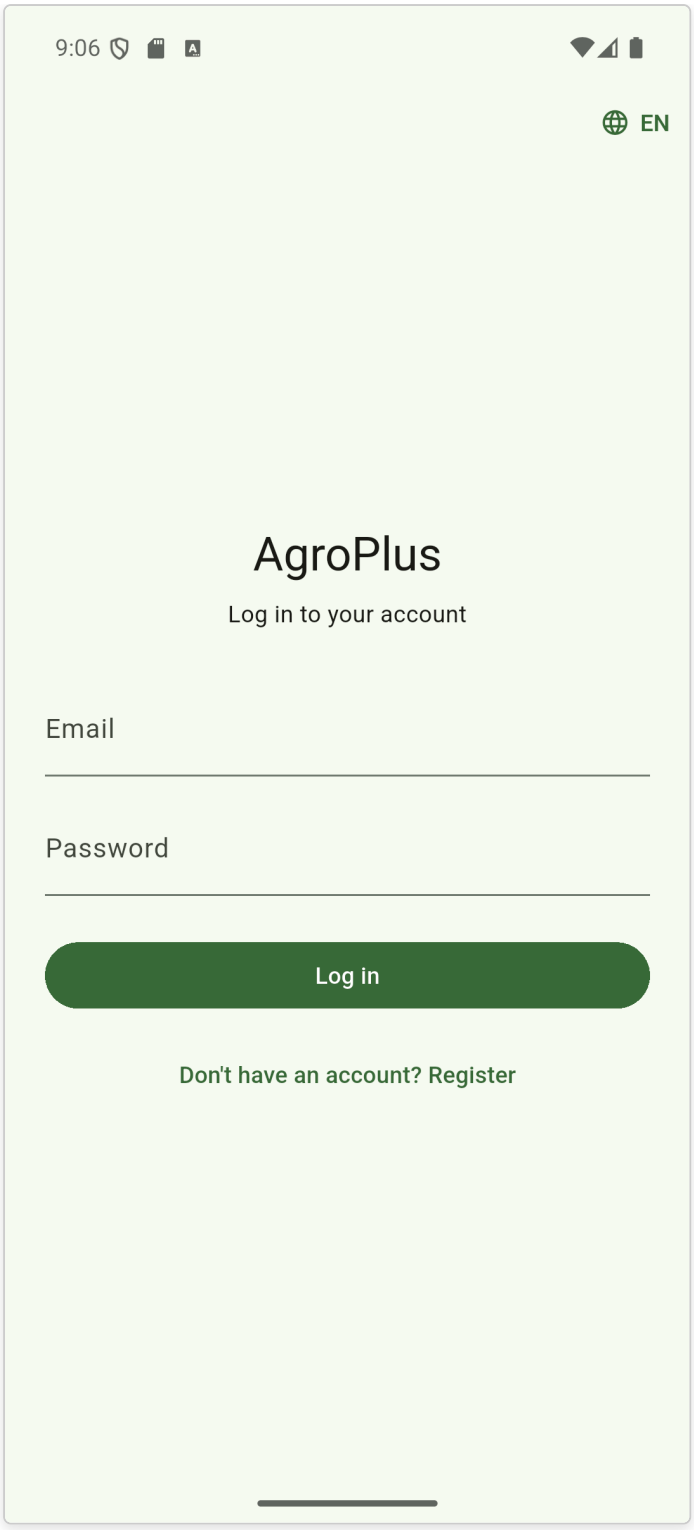


Figure 3: Login Screen

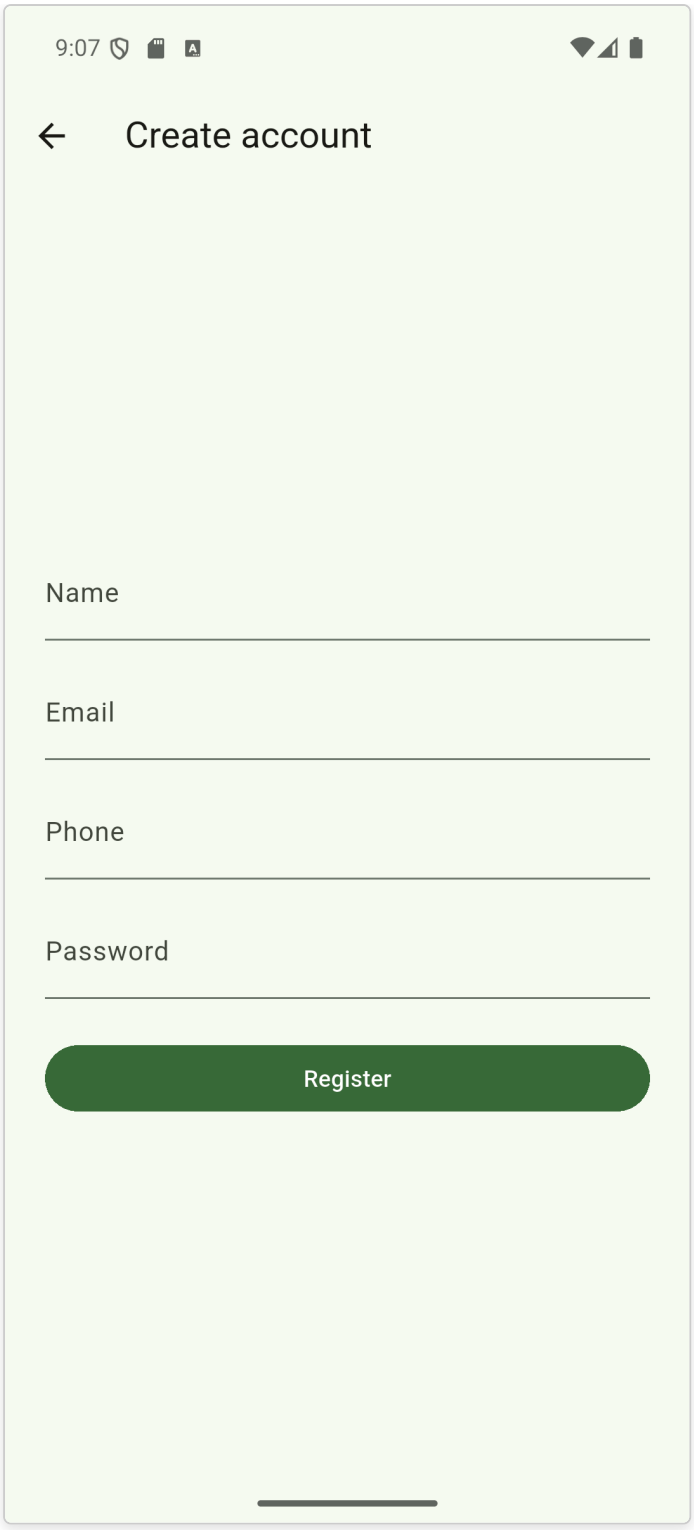


Figure 4: Create Account Screen

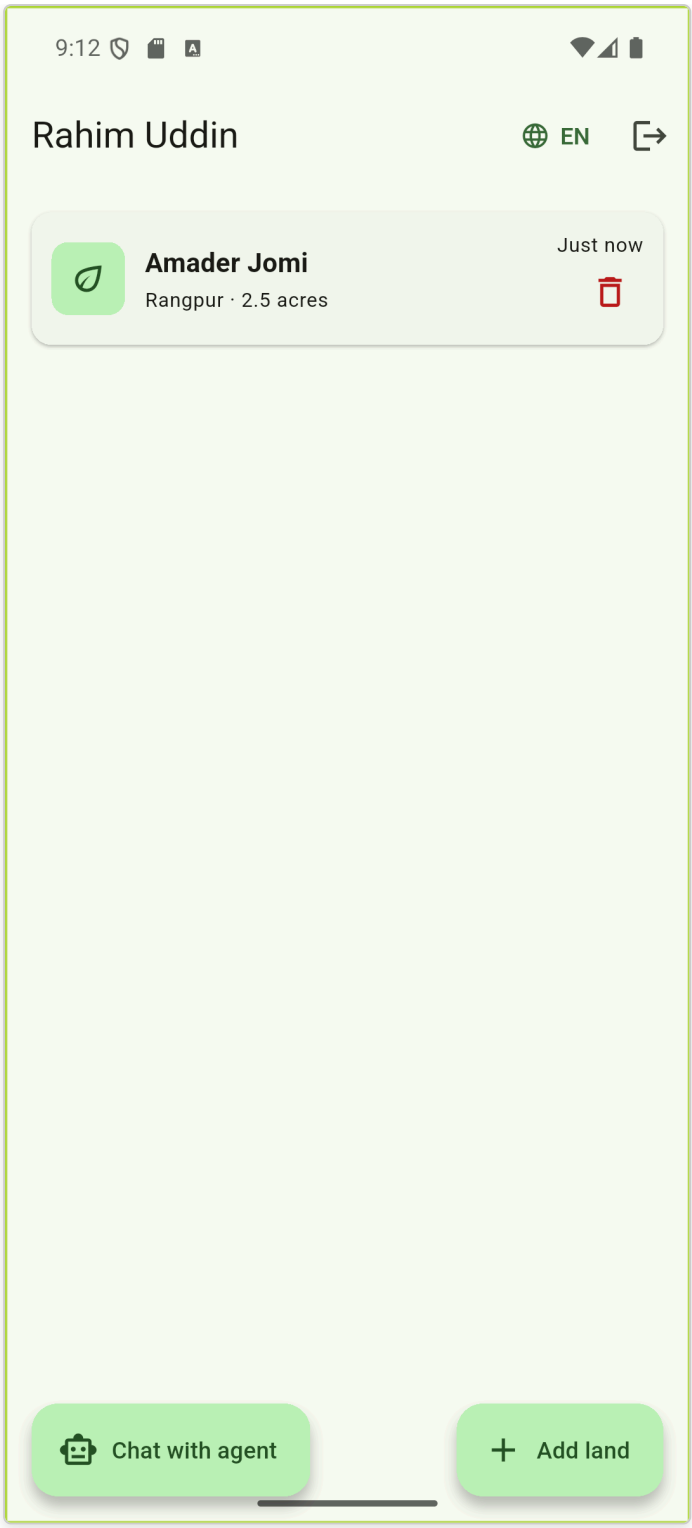


Figure 5: Dashboard — Land List

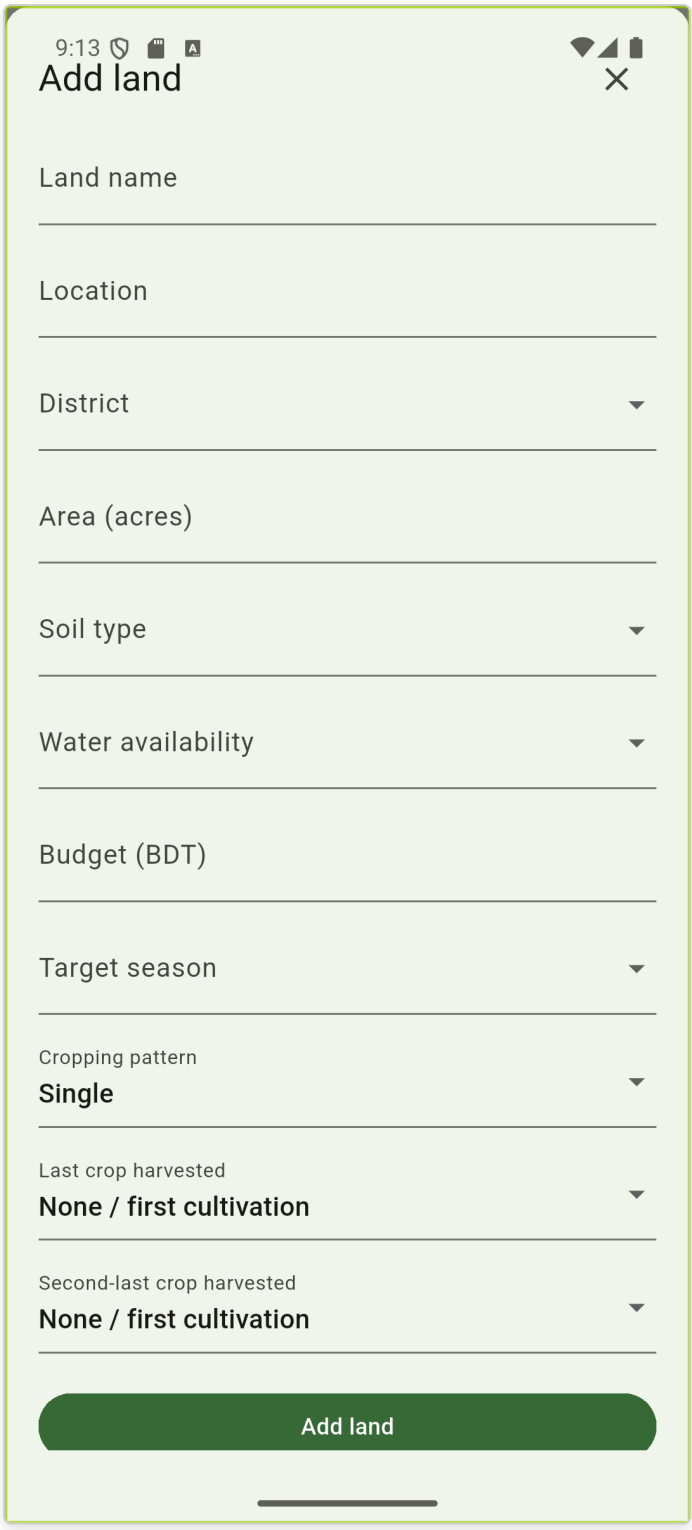


Figure 6: Add/Edit Land Form — including cropping pattern and crop-rotation history fields

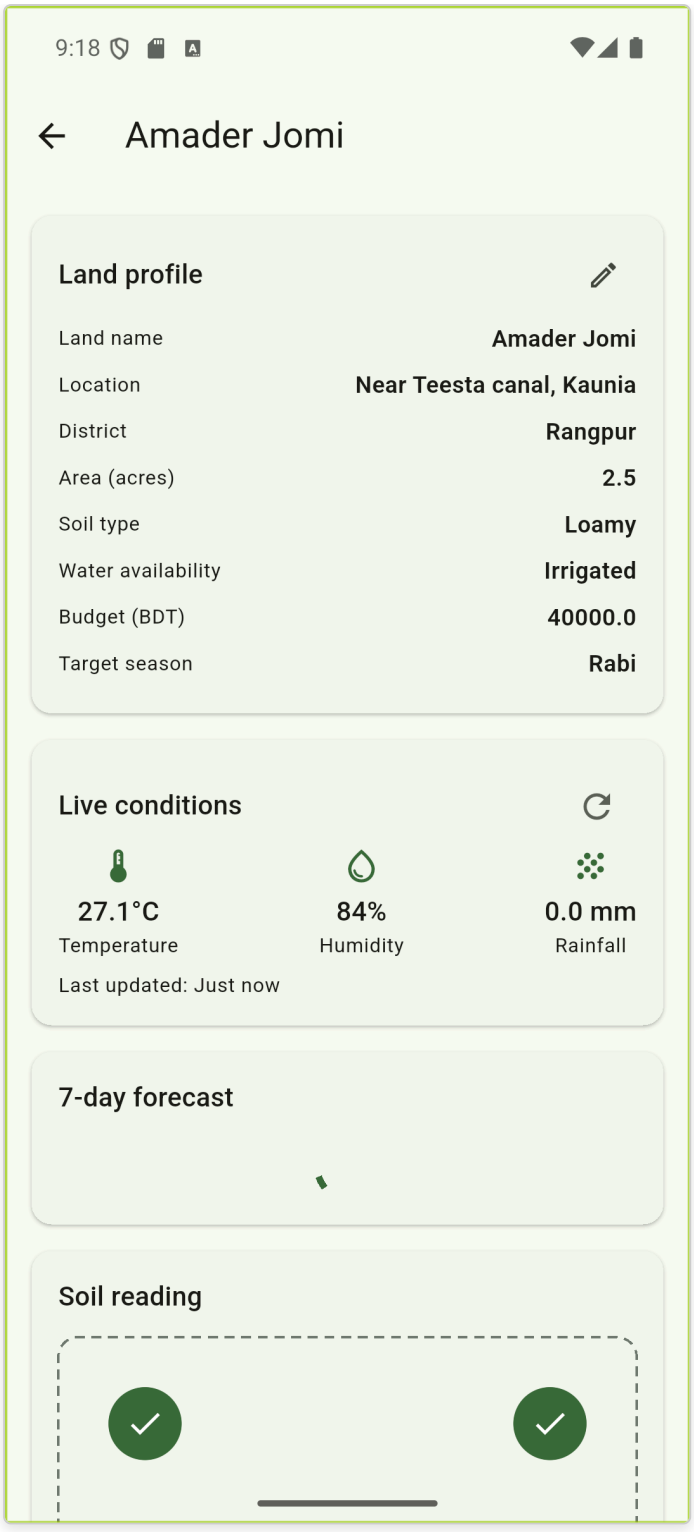


Figure 7: Land Profile — Details, Live Conditions, and Soil Reading Grid

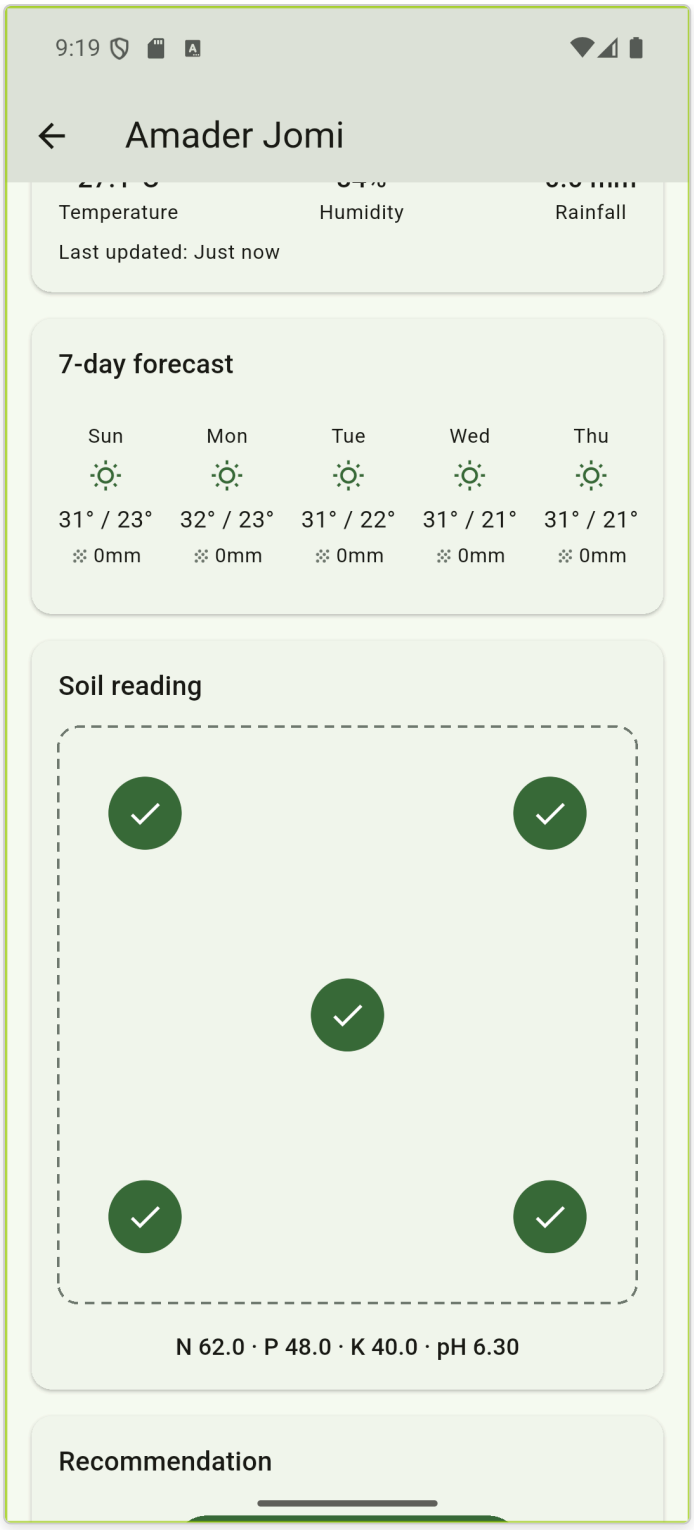


Figure 8: Land Profile — 7-Day Forecast and a Completed 5-Point Soil Sampling Grid

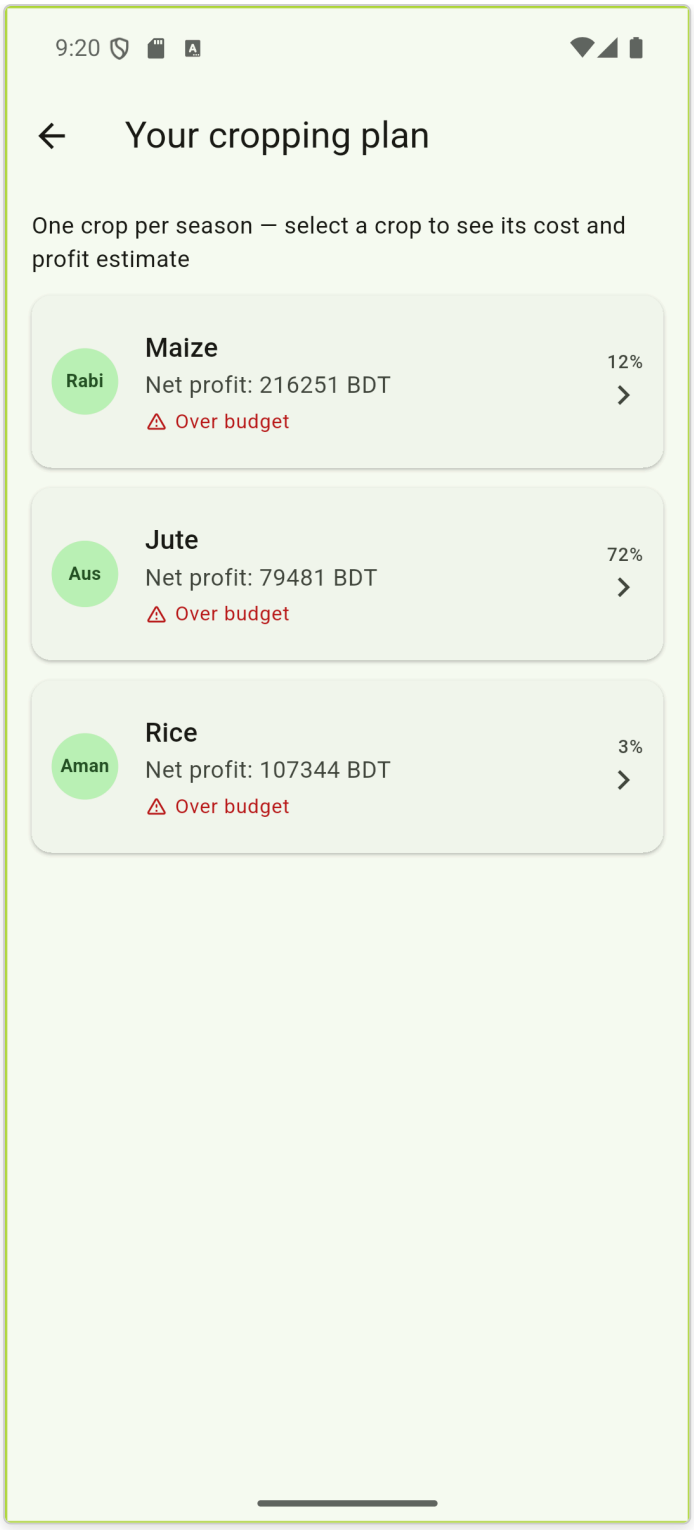


Figure 9: Triple-Cropping Plan — season badges (Rabi/Aus/Aman) with over-budget warnings

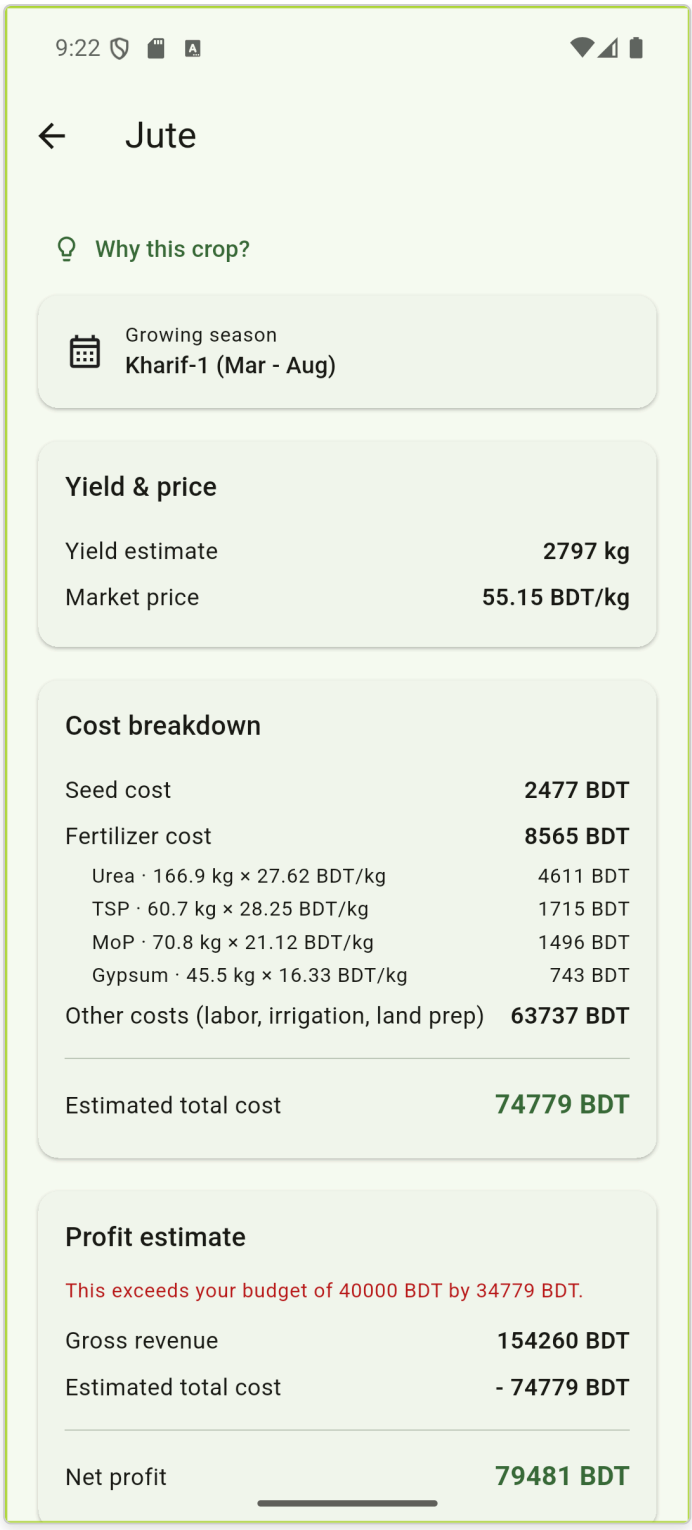


Figure 10: Crop Harvest Detail — itemized seed/fertilizer/other cost breakdown and profit estimate

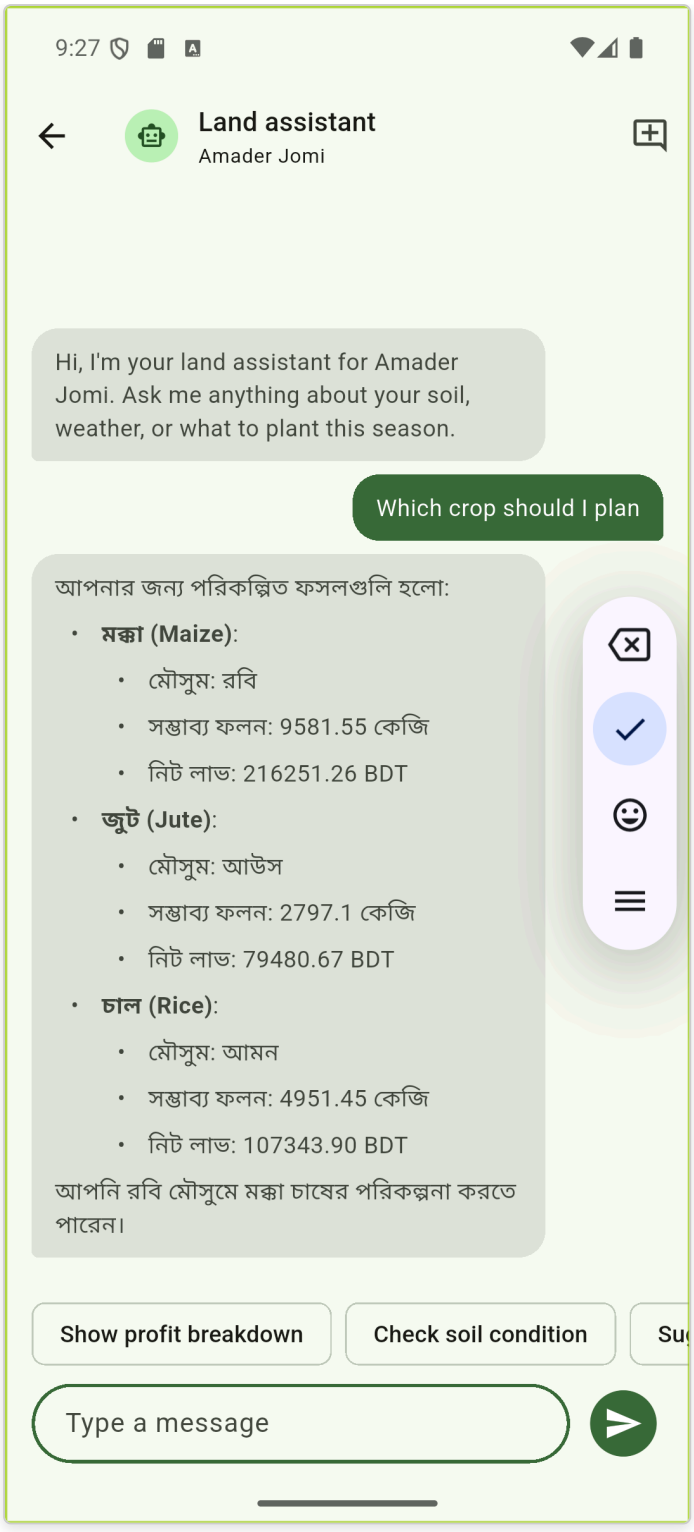


Figure 11: AI Farm Assistant — a question asked in English, answered in Bangla, summarizing the land's full cropping plan with figures grounded in real tool calls

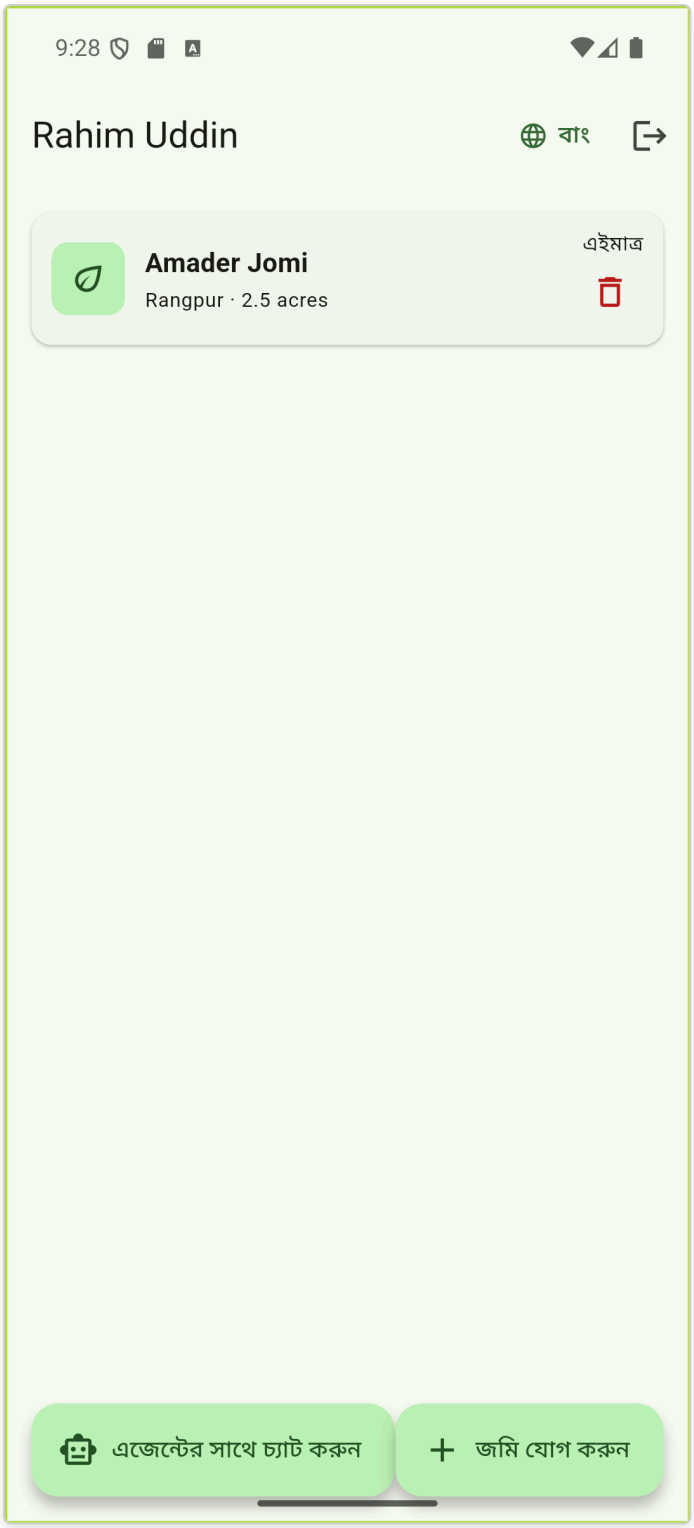


Figure 12: Dashboard in Bangla — full UI localization

8. Behavioural Modelling: State Transition Diagrams

State transition diagrams model how an interactive component moves between a finite set of states in response to user events and system responses. Each arrow is labeled with the triggering event and, where relevant, its side effect. The three diagrams below cover the major stateful flows of the application: authentication, the soil-sampling → recommendation pipeline, and chat.

8.1 AuthGate

States: checkingSession, login, register, authenticated.

On app start, the gate checks local secure storage for a stored access/refresh token pair and moves to *authenticated* if present, or *login* otherwise. From *login*, valid credentials lead to *authenticated*; a link tap moves to *register*. From *register*, a successful submission also lands in *authenticated* (AgroPlus issues tokens immediately on registration — there is no email-verification step in this iteration). *Logout* from *authenticated* clears the stored session and returns to *login*. A 401 response on any authenticated request silently attempts a token refresh before falling back to *login*.

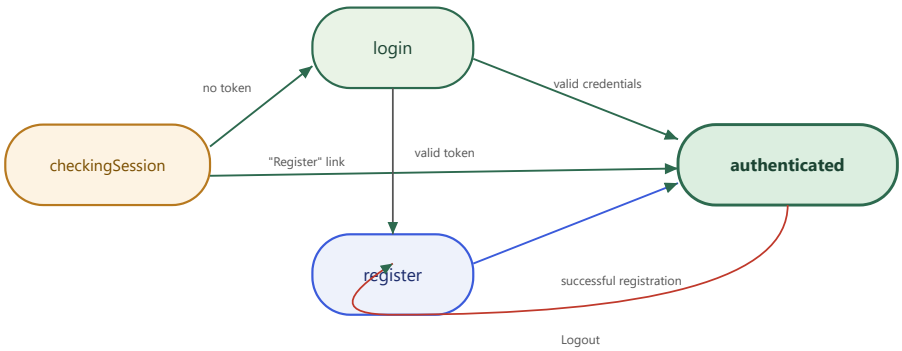


Figure 13: AuthGate State Transition

8.2 Soil Sampling & Recommendation Pipeline

States: incomplete, savingPoint, generating, ready, weatherUnavailable.

A land's sampling grid starts in *incomplete*. Saving any one point moves briefly through *savingPoint* and back to *incomplete* if the tier isn't yet fully satisfied. Once the last required point is saved, the pipeline moves to *generating*: it recomputes the composite reading and calls the weather provider. A successful weather fetch leads to *ready*, where the recommendation (single top-3 or a season plan, depending on `cropping_pattern`) is shown. A weather-provider failure leads to *weatherUnavailable* — the composite reading is still saved, but no crops are generated until the farmer re-saves a point to retry. Re-editing any point while already *ready* re-enters *generating* and discards the previous crop list, so results never go stale.

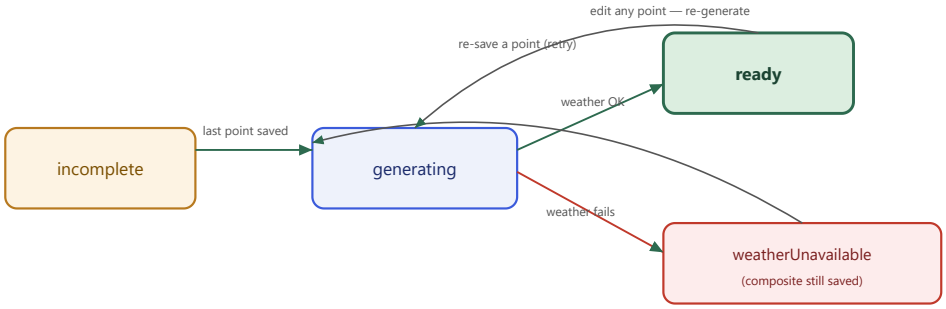


Figure 14: Soil Sampling & Recommendation Pipeline State Transition

8.3 Chat

States: loadingHistory, idle, sending, awaitingReply, confirmingReset.

Opening a land's chat starts in *loadingHistory*, fetching any saved thread; it resolves to *idle* showing either the restored conversation or, if none exists, the one-time greeting. Sending a non-empty message appends the farmer's bubble and moves through *sending* → *awaitingReply* (typing indicator shown) back to *idle* once the agent's reply appends. Tapping a quick-suggestion chip follows the same path with pre-filled text. Tapping "New conversation" moves to *confirmingReset*; on confirmation the thread is cleared server-side and the view resets to *idle* with just the greeting; cancelling returns to *idle* unchanged.

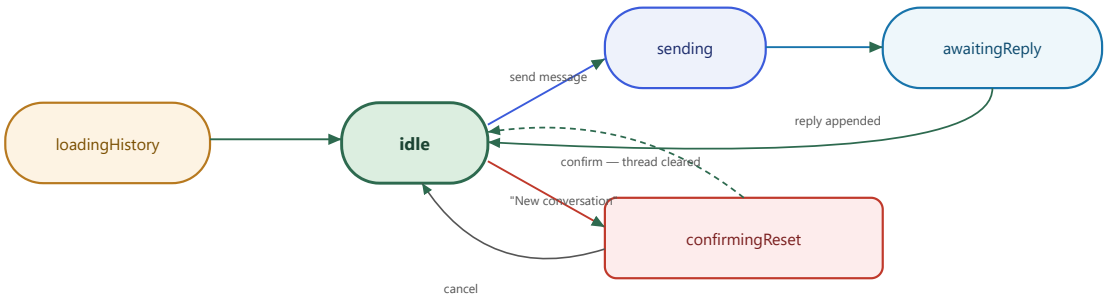


Figure 15: Chat State Transition

9. Testing: API Endpoint Testing

AgroPlus does not yet have an automated regression test suite (see §11, Future Work). Every endpoint listed below was instead verified by direct, repeatable testing against the live running backend during development — using scripted HTTP requests (Python `urllib`) for data-level correctness, cross-checked against real on-device interaction through the Flutter app (via `adb` UI automation) for end-to-end behaviour. All test cases listed below passed against the implemented system.

9.1 Auth — /auth

Test ID	Method	Objective	Expected Output
T-AU-01	`POST /auth/register`	Verify a new farmer can register with a valid name, email, phone, and password	201 Created; response body's email matches the submitted email; access and refresh tokens returned
T-AU-02	`POST /auth/register`	Verify registering with an email already in use is rejected	400 Bad Request — "Email already registered"
T-AU-03	`POST /auth/login`	Verify a registered farmer can log in with correct credentials	200 OK; body contains non-empty access_token and refresh_token
T-AU-04	`POST /auth/login`	Verify login with a wrong password is rejected	401 Unauthorized — "Invalid email or password"
T-AU-05	`POST /auth/refresh`	Verify a valid refresh token issues a new access token	200 OK; new access_token returned, distinct from the original
T-AU-06	`POST /auth/refresh`	Verify an access token (wrong type) is rejected by the refresh endpoint	401 Unauthorized
T-AU-07	`GET /farmers/me`	Verify a valid bearer token returns the caller's own profile	200 OK; name/email match the logged-in farmer
T-AU-08	`GET /farmers/me`	Verify the endpoint rejects a request with no Authorization header	401 Unauthorized

9.2 Districts — /districts

Test ID	Method	Objective	Expected Output

T-DI-01	`GET /districts`	Verify the full seeded district list is returned, sorted by name	200 OK; 64 districts; each with id, name, and (for most) center_lat/center_lng
T-DI-02	`GET /districts`	Verify the endpoint requires authentication	401 Unauthorized without a bearer token

9.3 Land Profiles — /farms

Test ID	Method	Objective	Expected Output
T-FA-01	`POST /farms`	Verify a land can be created with all required fields, including cropping_pattern and previous_crop_1/2	201 Created; response echoes every submitted field, including the new rotation fields
T-FA-02	`POST /farms`	Verify creating a land with an unknown district_id is rejected	400 Bad Request — "Unknown district_id"
T-FA-03	`GET /farms`	Verify the list endpoint returns only the authenticated farmer's own lands	200 OK; every returned farm's owner matches the caller
T-FA-04	`GET /farms/{id}`	Verify fetching another farmer's land is rejected	404 Not Found (ownership never disclosed)
T-FA-05	`PATCH /farms/{id}`	Verify a partial update (e.g. only cropping_pattern) leaves other fields unchanged	200 OK; only the submitted field differs from the prior state
T-FA-06	`PATCH /farms/{id}`	Verify editing size_acres across a sampling-tier boundary discards an in-progress reading	200 OK; the land's in_progress Recommendation (if any) is deleted; a completed one is untouched
T-FA-07	`DELETE /farms/{id}`	Verify deleting a land also removes its recommendations, sample points, and chat history (cascade)	204 No Content; subsequent GET on the land and its chat both 404
T-FA-08	`GET /farms/{id}/conditions`	Verify current weather is returned for a land with a district that has coordinates	200 OK; temperature, humidity, rainfall, fetched_at present

T-FA-09	`GET /farms/{id}/forecast`	Verify a 7-entry daily forecast is returned	200 OK; list of 7 {date, temp_max_c, temp_min_c, precipitation_mm}

9.4 Soil Sampling & Recommendations — /farms/{id}/recommendations

Test ID	Method	Objective	Expected Output
T-RE-01	`PUT .../points/{label}`	Verify submitting a point label outside the land's current tier is rejected	400 Bad Request, listing the valid labels for that tier
T-RE-02	`PUT .../points/{label}`	Verify saving the last required point for a tier completes the reading and generates a top-3 (single-cropping land)	200 OK; status: "complete"; composite_n/p/k/ph populated; 3 RecommendationCrop rows with season = null
T-RE-03	`PUT .../points/{label}`	Verify the same action on a triple-cropping land produces a season-sequenced plan instead	200 OK; up to 3 RecommendationCrop rows, each with a non-null season (Rabi/Aus/Aman), rank as plan order
T-RE-04	—	Verify a crop present twice in the land's previous_crop_1/previous_crop_2 is excluded from every slot of a new recommendation	The blocked crop never appears in top_crops; for a plan, the affected season shows a substituted crop with a non-null rotation_note
T-RE-05	`GET .../current`	Verify the most recent recommendation for a land is returned	200 OK (or null if none exists yet)
T-RE-06	`GET .../recommendations`	Verify history returns only completed recommendations, newest first	200 OK; list ordered by created_at descending; in_progress rows excluded
T-RE-07	`GET .../recommendations/{recId}`	Verify full detail (composite reading + every crop) of a past recommendation is returned	200 OK; points and top_crops arrays populated
T-RE-08	`GET .../crops/{crop}`	Verify the harvest detail endpoint returns an itemized cost breakdown that reconciles to the total	200 OK; seed_cost + fertilizer_cost + other_cost == total_cost (within rounding)
T-RE-09	`GET .../crops/{crop}`	Verify requesting a crop with no economics row returns a clear not-found rather than a fabricated figure	404 Not Found — "No economics data for this crop"

9.5 AI Farm Assistant — /farms/{id}/chat

Test ID	Method	Objective	Expected Output
T-CH-01	`GET .../chat/messages`	Verify a land with no prior conversation	200 OK; []

		returns an empty list	
T-CH-02	`POST .../chat`	Verify asking about soil condition with no saved reading yields an honest "unavailable" answer, not a fabricated number	200 OK; reply states no soil reading exists yet, prompts the farmer to add one
T-CH-03	`POST .../chat`	Verify asking "why this crop" returns figures that match the land's actual soil/weather values	200 OK; reply cites the real N/P/K/pH values present in the database for that land
T-CH-04	`POST .../chat`	Verify asking for crop options beyond the top-3 returns all eligible supported crops ranked by confidence, honoring rotation exclusions	200 OK; reply lists up to 8 crops minus any rotation-blocked and minus mango-if-not-applicable logic
T-CH-05	`POST .../chat`	Verify a second message in the same land reuses the persisted history as context	200 OK; GET .../chat/messages afterward shows both turns in order, role alternating user/assistant
T-CH-06	`DELETE .../chat/messages`	Verify clearing history removes all messages for that land only, not other lands	204 No Content; subsequent GET .../chat/messages for this land returns []; another land's history is untouched
T-CH-07	`POST .../chat`	Verify a request for a land owned by a different farmer is rejected	404 Not Found

9.6 Health Check

Test ID	Method	Objective	Expected Output
T-HE-01	`GET /health`	Verify the service liveness endpoint responds without depending on auth or DB state	200 OK; {"status": "ok"}

10. User Manual

This section explains how to use AgroPlus as an end user. It assumes no technical background — only an Android or iOS phone.

10.1 Creating an Account

- Open AgroPlus. If you don't have an account, tap "Don't have an account? Register" on the login screen.
- Enter your name, email, phone number, and a password, then tap "Register".
- You are logged in immediately and taken to your dashboard.

10.2 Logging In

- Enter your email and password, then tap "Log in".
- Once logged in, your name appears at the top of the dashboard along with a language toggle and a logout button.

10.3 Adding a Land

- From the dashboard, tap "Add land".
- Fill in the land's name, location, district, area in acres, soil type, water availability, budget, and target season.
- Choose a cropping pattern — Single for one crop at a time, or Double/Triple if you want a sequenced plan across two or three growing seasons.
- If you know what was grown on this land most recently, select it under "Last crop harvested" and, if applicable, "Second-last crop harvested" — this lets AgroPlus avoid suggesting the same crop too many times in a row.
- Tap "Add land" to save.

10.4 Taking a Soil Sample

- Open a land, scroll to "Soil reading". The number of points shown depends on the land's size — one for a very small plot, up to five for a larger one.
- Tap each point and enter its Nitrogen, Phosphorus, Potassium, and pH values from your soil test, then "Save".
- Once every point is filled, AgroPlus automatically computes a composite reading and generates your recommendation.

10.5 Viewing Your Crop Recommendation

- Tap "Best crops for my land" on the Recommendation card.
- For a Single-cropping land, you'll see the top-3 recommended crops ranked by confidence.
- For a Double or Triple-cropping land, you'll instead see "Your cropping plan" — one crop per growing season, labeled Rabi/Aus/Aman.
- Tap any crop to see its full breakdown: growing season, expected yield, market price, itemized cost (seed, fertilizer by nutrient, other costs), total cost, gross revenue, and net profit. A red warning appears if the cost exceeds your stated budget.
- Tap "Why this crop?" to ask the farm assistant to explain the recommendation in plain language.

10.6 Checking Weather

- A land's profile page shows current temperature, humidity, and rainfall under "Live conditions" (tap the refresh icon to update).
- Below it, "7-day forecast" shows the coming week's high/low temperatures and expected rainfall.

10.7 Chatting with the Farm Assistant

- Tap "Chat with agent" from the dashboard (you'll be asked which land) or directly from a land's page.
- Type a question, or tap one of the quick-suggestion chips ("Show profit breakdown", "Check soil condition", "Suggest another crop").
- The assistant answers using your land's real saved data — it will tell you plainly if something (like a soil reading) hasn't been added yet, rather than guessing.
- Your conversation is saved automatically; leaving and returning to the chat picks up where you left off.
- Tap the "New conversation" icon in the top bar to clear the history and start fresh.

10.8 Viewing Recommendation History

- Tap "View history" on the Recommendation card to see every past completed recommendation for that land, newest first.
- Tap any entry to see its full detail exactly as it appeared at the time, including season badges if it was a multi-cropping plan.

10.9 Editing or Deleting a Land

- Tap the pencil icon on a land's profile to edit any field, including cropping pattern and crop-rotation history.

- Tap the trash icon on the dashboard to delete a land — this permanently removes it and all its data, after confirmation.

10.10 Switching Language

- Tap the globe icon / "EN" or "বাং" label at the top of the dashboard to switch the entire app between English and Bangla at any time.

11. Conclusion & Future Work

11.1 Summary

AgroPlus delivers a complete farmer-facing loop: register, describe a piece of land, sample its soil, receive a crop recommendation grounded in a trained ML model and real government economics data, see exactly what that crop will cost and earn against a stated budget, and ask a conversational assistant to explain or extend that recommendation — all enforcing a basic agronomic rotation rule and available in both English and Bangla. Every monetary figure the system states, whether in a static screen or a chat reply, is traceable to a named public data source or a specific backend computation, never an invented estimate.

11.2 Known Limitations & Future Work

Area	Current State & Planned Extension
Crop coverage	Only 8 of the 22 crops the underlying ML model can recognize have sourced economics data and are ever recommended. Extending coverage means sourcing BAMIS/DAM/BADC data for the remaining crops.
Perennial crops in multi-cropping	Mango is excluded from the double/triple planner since its fertilizer recipe is priced per tree, not per acre, and it doesn't fit a within-year seasonal rotation. A tree-density figure would be needed to include it.
Season-specific climate data	The multi-cropping planner scores every season against the same current weather snapshot rather than season-specific historical normals per district — a known, documented approximation (§6.2.4).
Recommendation explanation depth	"Why this crop?" compares actual values against a crop's typical range plus global feature importance, not a true per-instance explanation (e.g. SHAP) — a deliberate scope/dependency trade-off for this iteration.
Automated testing	No unit or integration test suite exists yet; all testing (§9) was manual, scripted API verification plus on-device checks. A pytest suite with a disposable test database is the natural next step.
Farmer profile / settings	There is no screen to change a password or edit profile details after registration.
AI assistant cost controls	Each chat turn is an unmetered OpenAI API call; no per-farmer rate limiting or usage cap exists yet.
Production deployment	The backend runs only via a local development server (uvicorn --reload) against a local PostgreSQL instance; no hosting, CI/CD, or production secrets management has been set up.
iOS verification	The app has only been built and tested on Android throughout this project; the iOS build path is unverified.

None of these gaps block the delivered functionality described in this report — every feature in §1.3's "In Scope" list is built, persisted, and verified working end-to-end on a real device, as shown by the screenshots in §7.3.